



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 15
по курсу «Языки и методы программирования»
«MediaPipe. OpenCV. Компьютерное зрение»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Обзор проекта	1
Установка зависимостей	1
Ход работы	2
Лабораторная 1: Hand Tracking	2
Лабораторная 2: Pose Estimation	2
Лабораторная 3: ArUco Markers	3
Лабораторная 4: TCP Image Transfer	3
Лабораторная 5: HTTP MJPEG Streaming + Hands	3
Лабораторная 6: TCP Video Relay	3
Лабораторная 7: Phone Camera Source	3
Лабораторная 8: Ball Tracking	4
Лабораторная 9: Face Detection	4
Лабораторная 10: FaceMesh	4
Лабораторная 11: Objectron (3D)	4
Лабораторная 12: Holistic	4
Лабораторная 13: Face Recognition	4
Лабораторная 14: Server-Side ML	5
Сборка и запуск	5
Вывод	6

Цель работы

Изучить возможности фреймворка MediaPipe и библиотеки OpenCV для решения задач компьютерного зрения: детекция рук, позы, лиц, 3D-объектов, передача видео по сети, трекинг объектов и распознавание лиц.

Обзор проекта

Проект состоит из 14 последовательных лабораторных работ, охватывающих трекинг рук (Hands), оценку позы (Pose), детекцию ArUco-маркеров, передачу изображений и видео по TCP/HTTP, трекинг мяча, детекцию и разметку лиц, 3D-детекцию объектов (Objectron), комплексный Holistic-анализ и распознавание лиц.

Стек технологий: Python 3.12, MediaPipe 0.10.21, OpenCV, Pillow, NumPy, face_recognition

Установка зависимостей

Зависимости управляются через uv. Основные пакеты:

pyproject.toml – фрагмент

```
[project]
dependencies = [
    "mediapipe==0.10.21",
    "face-recognition==1.2.3",
]
```

Ход работы

Лабораторная 1: Hand Tracking

Обнаружение 21 ключевой точки кисти руки с помощью `mp.solutions.hands`. Кадр с веб-камеры переводится в RGB, подаётся в MediaPipe, результат отрисовывается поверх изображения.

1_Hands_StafNum_1/hands.py

```
import cv2
import mediapipe as mp

mp_hands = mp.solutions.hands
mp_draw = mp.solutions.drawing_utils
hands = mp_hands.Hands()

cap = cv2.VideoCapture(0)
while cap.isOpened():
    success, frame = cap.read()
    if not success:
        continue
    frame = cv2.flip(frame, 1)
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    result = hands.process(rgb)
    if result.multi_hand_landmarks:
        for hand_landmarks in result.multi_hand_landmarks:
            mp_draw.draw_landmarks(
                frame, hand_landmarks,
                mp_hands.HAND_CONNECTIONS)
    cv2.imshow("Hand Tracking", frame)
    if cv2.waitKey(1) & 0xFF == 27:
        break
```

Лабораторная 2: Pose Estimation

Оценка позы человека (33 ключевые точки) с помощью `mp.solutions.pose`. Аналогичный пайплайн с веб-камеры, отрисовка скелета с соединениями `POSE_CONNECTIONS`.

Лабораторная 3: ArUco Markers

Детекция ArUco-маркеров (словарь 6x6, ID 0-999) с использованием `cv2.aruco`. Чисто OpenCV-решение без MediaPipe.

3_Aruco_StafNum_3/aruco.py

```
import cv2

aruco_dict = cv2.aruco.getPredefinedDictionary(
    cv2.aruco.DICT_6X6_1000)
params = cv2.aruco.DetectorParameters()
detector = cv2.aruco.ArucoDetector(aruco_dict, params)

cap = cv2.VideoCapture(0)
while cap.isOpened():
    _, frame = cap.read()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, _ = detector.detectMarkers(gray)
    if ids is not None:
        cv2.aruco.drawDetectedMarkers(frame, corners, ids)
    cv2.imshow("ArUco", frame)
    if cv2.waitKey(1) & 0xFF == ord(' '):
        break
```

Лабораторная 4: TCP Image Transfer

Передача изображения с камеры на сервер по TCP-сокету. Сервер (`serv.py`) принимает JPEG-байты на порту 9092 и сохраняет их через Pillow. Клиент (`client_webcam.py`) захватывает кадр с веб-камеры и отправляет его на сервер.

Лабораторная 5: HTTP MJPEG Streaming + Hands

Веб-сервер на встроенном `http.server`, который в фоновом потоке захватывает видео с веб-камеры, применяет MediaPipe Hands и отдаёт MJPEG-поток на `:8000/stream.mjpg`. Браузер отображает видео с наложенными ключевыми точками кистей.

Лабораторная 6: TCP Video Relay

Архитектура клиент-сервер: клиент захватывает видео, применяет Hands, сериализует кадры через `pickle` и отправляет по TCP на сервер. Сервер принимает поток и транслирует через HTTP MJPEG.

Лабораторная 7: Phone Camera Source

Подключение камеры смартфона через IP Webcam (Android). Видеозахват через `cv2.VideoCapture("http://IP:8080")` и трансляция через HTTP MJPEG.

Лабораторная 8: Ball Tracking

Классический трекинг объекта по цвету (HSV-фильтрация зелёного мяча).

Контурный анализ, вычисление центра и радиуса, построение траектории, экспорт данных в CSV и визуализация графика угла поворота.

8_ball_tracking_StafNum_9/ball.py — ключевые шаги

```
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, greenLower, greenUpper)
mask = cv2.erode(mask, None, iterations=2)
mask = cv2.dilate(mask, None, iterations=2)

contours, _ = cv2.findContours(mask,
                                cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
if contours:
    c = max(contours, key=cv2.contourArea)
    ((x, y), radius) = cv2.minEnclosingCircle(c)
    # трекинг + логирование
```

Лабораторная 9: Face Detection

Детекция лиц с помощью `mp.solutions.face_detection`. Вывод bounding box вокруг каждого обнаруженного лица.

Лабораторная 10: FaceMesh

Построение сетки лица (468 ключевых точек) с помощью `mp.solutions.face_mesh`. Отрисовка `FACEMESH_TESSELATION` — плотной треугольной сетки.

Лабораторная 11: Objectron (3D)

3D-детекция объектов (модель Chair) с помощью `mp.solutions.objectron`. Вывод 2D- и 3D-ограничивающих прямоугольников, осей координат и 3D-ключевых точек.

Лабораторная 12: Holistic

Комбинированный анализ: `mp.solutions.holistic` одновременно детектирует лицо (468 точек), левую и правую кисти (по 21 точке) и позу (33 точки) на одном кадре.

Лабораторная 13: Face Recognition

Распознавание лиц с помощью библиотеки `face_recognition`. Загрузка эталонных изображений, вычисление 128-мерных эмбеддингов, сравнение с веб-камеры в реальном времени.

13_face_recognizer_StafNum_18/face_recognizer.py

```
import face_recognition

obama_img = face_recognition.load_image_file("obama.jpg")
obama_enc = face_recognition.face_encodings(obama_img)

me_img = face_recognition.load_image_file("i.jpg")
me_enc = face_recognition.face_encodings(me_img)

cap = cv2.VideoCapture(0)
while cap.isOpened():
    _, frame = cap.read()
    small = cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)
    rgb = cv2.cvtColor(small, cv2.COLOR_BGR2RGB)
    face_locs = face_recognition.face_locations(rgb)
    face_encs = face_recognition.face_encodings(rgb, face_locs)
    for enc, loc in zip(face_encs, face_locs):
        matches = face_recognition.compare_faces(
            known_encodings, enc, tolerance=0.6)
```

Лабораторная 14: Server-Side ML

MediaPipe Hands вынесен на серверную сторону. Тонкий клиент отправляет сырые RGB-кадры по TCP, сервер выполняет ML-обработку и транслирует результат через HTTP MJPEG.

Сборка и запуск

В проекте используется make с пронумерованными целями:

Makefile — цели

```
1: uv run 1_Hands_StafNum_1/hands.py
2: uv run 2_Pose_StafNum_2/Pose.py
3: uv run 3_Aruco_StafNum_3/aruco.py
4: kill-server; uv run 4_.../serv.py & uv run 4_.../client.py
5: kill-server; uv run 5_.../webcam.py & sleep 10; kill-server
6: kill-server; uv run 6_.../server.py & uv run 6_.../clien.py
9: uv run 9_FaceDetection/face_detection.py
10: uv run 10_FaceMash/face_points.py
11: uv run 11_Objectron/chairs.py
12: uv run 12_Holistic/Holistic.py
13: cd 13_face_recognizer && uv run face_recognizer.py
```

Пример запуска:

Терминал

```
cd lab-ml
make 1    # Hand Tracking
make 5    # HTTP MJPEG + Hands
make 13   # Face Recognition
```

Вывод

В ходе цикла лабораторных работ освоены:

- MediaPipe (решения Hands, Pose, Face Detection, FaceMesh, Objectron, Holistic);
- OpenCV (видеозахват, HSV-фильтрация, контурный анализ, ArUco);
- сетевое взаимодействие (TCP-сокеты, HTTP MJPEG-стриминг, IP-камеры);
- распознавание лиц (библиотека face_recognition);
- ML-инференс на стороне сервера (тонкий клиент).

Все 14 лабораторных работ выполнены и протестированы с веб-камерой, IP-камерой телефона и эталонными изображениями.